ITPRA0 -22: Project

Site: Eduvos LMS
Course: Program Design Assessments
Book: ITPRA0 -22: Project
Printed by: Kgothatso Masha
Date: Monday, 28 September 2026, 9:32 AM

Table of contents

1. Project
2. Instructions to Students
3. Section A
    3.1. Question 1
    3.2. Marksheet
    3.3. Section B

1. Project

Faculty:	Information Technology
Module Code:	ITPRA0-22
Module Name:	Program Design
Content Writer:	Faith Mutepfe
Internal Moderation:	Ojong Ayuk
Copy Editor:	Phindile Tabata
Total Marks:	100
Submission Week:	6

This module is presented on NQF level 5.

5% will be deducted from the student's project mark for each calendar day the project is submitted late, up to a maximum of three calendar days. The penalty will be based on the official campus submission date.

Projects submitted later than three calendar days after the deadline or not submitted will get 0%. [1]

This is an individual project.

This project contributes 30% towards the final mark.

[1] Under no circumstances will projects be accepted for marking after the projects of other students have been marked and returned to the students.

2. Instructions to Students

1. Please ensure that your answer file (where applicable) is named as follows before submission: **Module Code – Assessment Type – Campus Name – Student Number.**
2. Remember to keep a copy of all submitted assignments.
3. All work must be typed.
4. Please note that you will be evaluated on your writing skills in all your assignments.
5. All work must be submitted through Turnitin. The full originality report will be automatically generated and available for the lecturer to assess. Negative marking will be applied if you are found guilty of plagiarism, poor writing skills, or if you have applied incorrect or insufficient referencing. (See the "instructions to students" book activity before this activity where the application of negative marking is explained.)
6. You are not allowed to offer your work for sale or to purchase the work of other students. This includes the use of professional assignment writers and websites, such as Essay Box. You are also not allowed to make use of artificial intelligence tools, such as ChatGPT, to create content and submit it as your own work. If this should happen, Eduvos reserves the right not to accept future submissions from you.

3. Section A

Learning Objectives

1. Define basic algorithms to carry out an operation and outline the process of programming an application
2. Explain the characteristics of procedural, object-orientated, and event-driven programming, and conduct an analysis of a suitable IDE
3. Design and implement basic algorithms in code using an IDE
4. Determine the debugging process and explain the importance of a coding standard

Project Topics

Week 1 - Week 7

Scope

1. Class design(OOP Principles)
2. Lists
3. Control structures

4. Exception Handling
5. Methods and Functions
6. Strings
7. Event Handling

3.1. Question 1

Scenario

Study the scenario and answer the questions that follow:

You have been hired by a local community learning centre to develop a Python-based Learner Progress Tracking System (LPTS). The system will help staff capture learner details, record assessment marks, determine learner performance, and generate simple progress feedback.

The centre offers short training programmes to learners of different ages. Each learner is registered for a course and completes several assessments. Staff members need a system that can store learner details, calculate averages, determine pass or fail status, and display progress summaries.

The application must be developed in Python and should demonstrate the use of fundamental programming concepts studied in the module, including variables, input/output, arithmetic operations, decision structures, loops, functions, lists, object-oriented programming, inheritance, encapsulation, recursion, exception handling, and simple GUI feedback using message boxes.

You are required to develop a Python application for the Learner Progress Tracking System described above.

Requirements

1. Learner Management

- Create a class called Learner with the following attributes:
  - learner_id
  - name
  - age
  - course
  - marks (stored in a list)
- The system must allow staff to:
  - Add new learners
  - View learner details
  - Update learner details
  - Remove learners from the system

2. Inheritance

- Create a base class called Person with common attributes such as:
  - name
  - age
- Create a derived class called Learner that inherits from Person.

3. Encapsulation

- At least one learner attribute, such as the average mark or learner status, must be protected using encapsulation.
- Access to this value must be controlled using methods.

4. Assessment and Marks

- Each learner must have a list of marks.
- The system must allow staff to:

    - Capture marks for a learner
    - Display all marks for a learner
    - Calculate the learner's average mark
- The system must determine whether the learner:
    - Passed
    - Failed
    - Passed with distinction

Suggested rules:

- Below 50: Fail
- 50 to 74: Pass
- 75 and above: Pass with distinction

5. Decision Structures

- Use if, if-else, and if-elif-else statements to:
    - Validate learner age
    - Check whether marks are valid
    - Determine learner performance category
    - Check whether a learner qualifies for a certificate

6. Repetition Structures

- Use loops in the program, including:
    - A condition-controlled loop for the main menu
    - A for loop to process learner marks
- Include the use of:
    - break
    - continue

7. Functions

Create functions to perform tasks such as:

- Adding a learner
- Entering marks
- Calculating average
- Displaying learner summaries
- Searching for a learner by ID

The program must also include at least one of these functions:

- A predicate function that returns True or False
- A function with a default argument
- A function call that uses a keyword argument

8. Recursion

- Include one recursive function.
- For example, create a recursive function that:
    - Calculates the sum of all marks in a learner's mark list
    - Or counts the number of learners in the system

9. Exception Handling

- Use try-except blocks to handle:
    - Invalid numeric input
    - Invalid mark entries
    - Division by zero where applicable
    - Menu selection errors

10. Lists

- Use a list to store learners.
- Use a list to store marks for each learner.
- The system must be able to:
  - Traverse the lists
  - Search through the lists
  - Update list values where necessary

11. GUI Requirement

- The system may be console-based, but it must include simple GUI feedback using message boxes.
- Use tkinter.messagebox to display messages such as:
  - Successful learner registration
  - Invalid input
  - Pass/fail result
  - Exit confirmation

Note: Full event-driven GUI design is not required. Only simple message boxes should be used where appropriate.

12. Output

The system must display:

- Learner details
- Marks entered
- Average mark
- Performance result
- Whether the learner qualifies for a certificate

13. Documentation

Your program must include:

- Comments at the top of the file indicating:
  - Programmer name
  - Date
  - Program purpose
- Comments for important sections of the code
- Meaningful variable names
- Proper indentation

Deliverable

Develop and submit a working **Python Learner Progress Tracking System** prototype for internal review, together with well-documented source code.

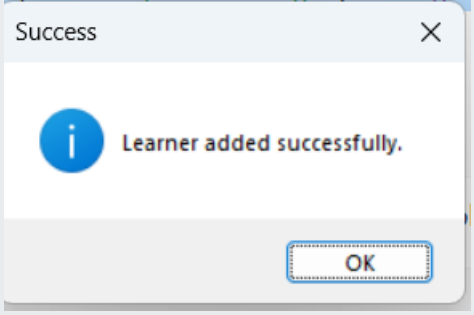
Example of working program

Adding a learner

```
In [1]: %runfile C:/Users/cash/.spyder-py3/temp.py --wdir

===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 1
Enter learner ID: 1
Enter learner name: lucy manyange
Enter learner age: 18
Enter course name: intro to python
```

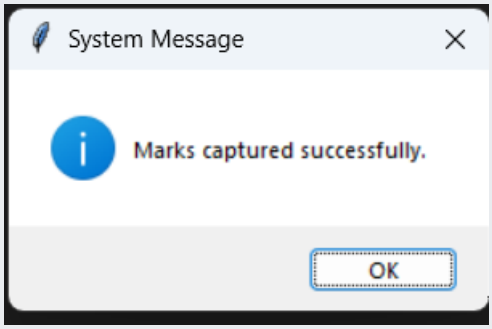
Messagebox showing learner successfully added



Adding Marks

```
===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 2
Enter learner ID: 1
How many marks do you want to enter? 3
Enter mark 1: 50
Enter mark 2: 76
Enter mark 3: 65
```

Messagebox showing marks added successfully



Viewing Students

```
Enter your choice: 3

----- Learner Summary -----
Learner ID: 1
Name: lucy manyange
Age: 18
Course: intro to python
Marks: [50.0, 76.0, 65.0]
Average: 63.67
Result: Pass
Certificate: Yes
```

Searching for a learner

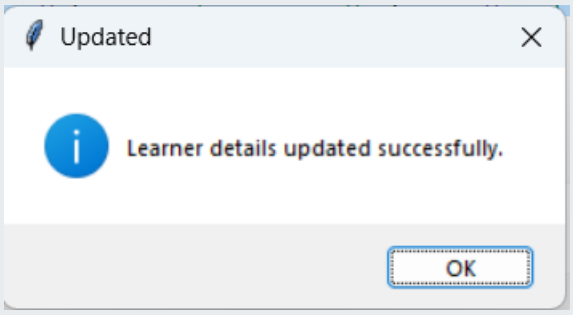
```
Enter your choice: 4
Enter learner ID to search: 1

----- Learner Summary -----
Learner ID: 1
Name: lucy manyange
Age: 18
Course: intro to python
Marks: [50.0, 76.0, 65.0]
Average: 63.67
Result: Pass
Certificate: Yes
```

Updating a learner

```
===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 5
Enter learner ID to update: 1
Enter new name: Gaby
Enter new age: 19
Enter new course: IT
```

Learner successfully updated

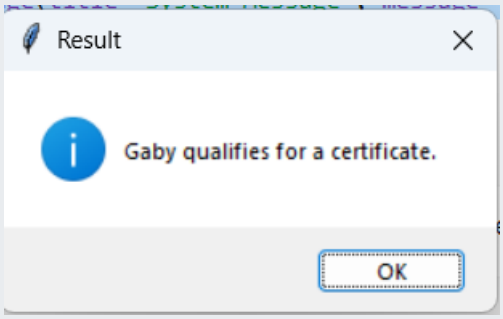


Showing learner results

```
7. Show learner result
8. Exit
Enter your choice: 7
Enter learner ID: 1

Result for Gaby
Average: 63.67
Performance: Pass
```

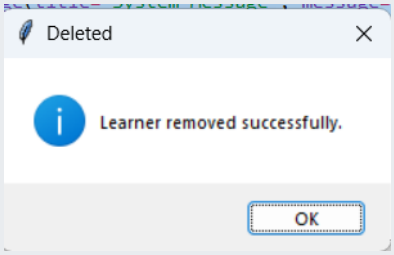
Showing whether student qualifies for certificate



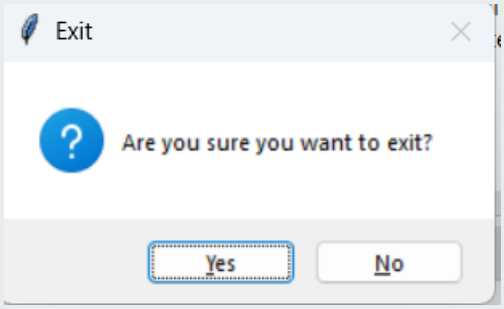
learner

```
===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 6
Enter learner ID to remove: 1
```

Learner deleted successfully



Exit



3.2. Marksheet

Achievement	% Possible	Achieved
Source code documentation		
Comments at top of file (name, date, purpose)	2	
Comments for important code sections	2	
Meaningful variable, function, and class names	2	
Correct indentation and formatting	2	
General readability and organisation	2	
Subtotal	**10**	
Program functionality		
Main menu displays and works correctly	3	
Adding a learner works correctly	4	

Viewing all learners works correctly	3	
Searching for a learner works correctly	3	
Updating learner details works correctly	4	
Removing a learner works correctly	3	
Capturing learner marks works correctly	4	
Marks stored correctly in a list	3	
Average calculated correctly	4	
Correct result displayed (Fail / Pass / Distinction)	4	
Certificate eligibility determined correctly	3	
Learner summaries displayed correctly	3	
Message boxes used appropriately	2	
Exit option works correctly	2	
Subtotal	**45**	
Decision structures and loops		
Correct use of if / if-else / if-elif-else	3	
Condition-controlled loop used for main menu	4	
For loop used correctly for marks processing	3	
Appropriate use of continue	2	
Appropriate use of break	2	
Subtotal	**14**	
Functions and modular design		
Program decomposed into logical functions	2	
Functions perform required tasks correctly	3	
Predicate/ Default argument / Keyword argument function implemented correctly	3	
Subtotal	**8**	
Object-oriented programming		
Person class implemented correctly	2	
Learner class inherits correctly from Person	3	
Objects created correctly	2	
Methods used appropriately	2	
Encapsulation applied correctly	3	
Subtotal	**12**	
Advanced concepts and error handling		
Recursive function implemented correctly	4	

Recursion solves an appropriate problem	2	
Exception handling for invalid numeric input	3	
Exception handling for menu and mark entry errors	2	
Subtotal	**11**	
General Comments		
Total	**100**	

3.3. Section B

Plagiarism and Referencing

Eduvos places high importance on honesty in academic work submitted by students, and adopts a policy of zero tolerance on cheating and plagiarism. In academic writing, any source material e.g. journal articles, books, magazines, newspapers, reference material (dictionaries), online resources (websites, electronic journals or online newspaper articles), must be properly acknowledged. Failure to acknowledge such material is considered plagiarism; this is deemed an attempt to mislead and deceive the reader, and is unacceptable. Eduvos adopts a zero tolerance policy on plagiarism, therefore, any submitted assessment that has been plagiarised will be subject to severe penalties. Students who are found guilty of plagiarism may be subject to disciplinary procedures and outcomes may include suspension from Eduvos or even expulsion. Therefore, students are strongly encouraged to familiarise themselves with referencing techniques for academic work. Students can access the Guide to Referencing on myLMS.